



Merge Sort

A trivial divide, a clever combine, and the first $\Theta(n \log n)$ algorithm of the course

Session 11 of 24 · Friday 23 October 2026 · 8:30 – 9:50 AM

· Prof. Guillaume Sanje Mpacko, Ph.D.

SESSION 11 OF 24

Merge Sort

Friday, 23 October 2026 · 8:30 – 9:50 AM

LEARNING OBJECTIVES

- ✓ State the divide-and-conquer template and locate it in unfamiliar code.
- ✓ Implement merging of two sorted runs and justify the linear bound.
- ✓ Derive merge sort's $\Theta(n \log n)$ from the recursion tree of Session 10.
- ✓ Explain stability, and why merge sort needs $\Theta(n)$ auxiliary memory.
- ✓ Adapt merge sort to count inversions without changing its complexity class.

FROM SESSIONS 9 AND 10

Insertion sort obeys $T(n) = T(n-1) + n = \Theta(n^2)$. We also solved $T(n) = 2T(n/2) + n$ by drawing it: every level costs n , there are $\log_2 n + 1$ levels, so $\Theta(n \log n)$. Today we meet the algorithm that second recurrence describes, and find out what the improvement costs.

1 · The Template

10 min

- 1 **Divide** the problem into subproblems of the same kind, usually of size $n/2$.

2 **Conquer** them by recursion, stopping when the pieces are small enough to solve outright.

3 **Combine** the sub-answers into an answer for the whole.

The cost follows immediately: $T(n) = a \cdot T(n/b) + (\text{divide cost} + \text{combine cost})$. Designing a divide-and-conquer algorithm is therefore a decision about **where to put the intelligence**.

Trivial divide, clever combine

Merge sort. Cutting a list in half takes no thought at all; all the work is in putting the halves back together correctly.

Clever divide, trivial combine

Quicksort, on 3 November. All the work goes into arranging the split so that the combine step is literally nothing.

The condition that must hold

The subproblems must be **independent**: nothing learned from the left half may be needed to solve the right one. When that fails — when subproblems overlap — you are in the Fibonacci disaster of Session 8, and divide and conquer is the wrong tool entirely.

2 · Merging Two Sorted Runs 20 min

Everything rests on one observation: **two sorted lists can be combined into one sorted list in linear time**. Sorting from scratch is expensive; merging things that are already sorted is cheap, and merge sort is built entirely out of that asymmetry.

left **2** **5** **9** **12** right **1** **6** **7** **20**

Compare the two front elements, move the smaller one out, advance that pointer. Repeat.

Each comparison removes exactly one element from consideration → at most $|left| + |right|$ comparisons.

merged **1** **2** **5** **6** **7** **9** **12** **20**

8 elements out, 8 moves in, no element examined twice — merging is $O(n)$.

The merge. Look only at the two front elements. The smaller of them is the smallest element remaining anywhere, because both lists are sorted — so it can be emitted at once and never reconsidered. Each comparison permanently places one element, so n elements need at most n comparisons.

```
def merge(left, right):
    """Combine two sorted lists into one sorted list."""
    out = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:          # <= keeps equal elements in their original order
            out.append(left[i])
            i += 1
        else:
            out.append(right[j])
            j += 1
    out.extend(left[i:])                # one side is exhausted; the other is already sorted
    out.extend(right[j:])
    return out
```

WHY <= AND NOT <

When the two front elements are equal, taking from the **left** preserves their original relative order. That property is called **stability**. It is what lets you sort a table of students by name, then by grade, and still have equal grades appear in name order. Changing one character here silently destroys it, and the failure will not show up in any test that only checks “is the output sorted?”

Trace it at the board

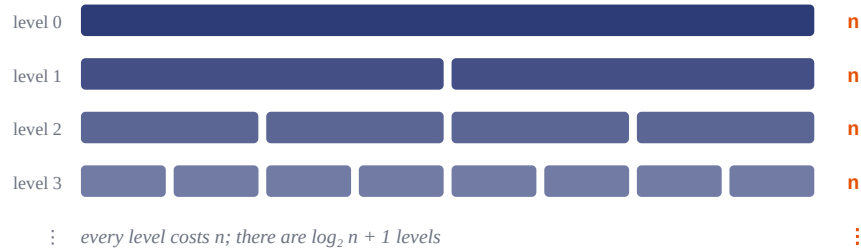
`merge([2, 5, 9, 12], [1, 6, 7, 20])` . Ask the class for the output one element at a time and record which pointer advanced. Eight outputs, seven comparisons, and the last `extend` moves the remaining tail with no comparisons at all.

3 · Merge Sort 15 min

```
def merge_sort(A):
    """Sort by dividing in half, sorting each half, and merging."""
    if len(A) <= 1:                    # a list of 0 or 1 elements is already sorted
        return A[:]
    mid = len(A) // 2
    left = merge_sort(A[:mid])
    right = merge_sort(A[mid:])
    return merge(left, right)
```

Read the recurrence off the code exactly as in Session 8: two recursive calls on halves, plus a linear merge. $T(n) = 2T(n/2) + n$ — the recurrence we have already solved twice.

$T(n) = 2T(n/2) + n$ — merge sort



The same tree, now with a meaning attached. Each bar is a sublist. At every level, each element is merged exactly once — so the level costs n no matter how many pieces the level is split across. **$\Theta(n \log n)$ in the best, worst and average case alike**, because the split is by position and never depends on the values.

What merge sort guarantees

$\Theta(n \log n)$ on every input — there is no bad case, because the algorithm never looks at the data when deciding how to split. Stable. Splits naturally across files, machines and threads, which is why it underlies external sorting of data too large for memory.

What it costs

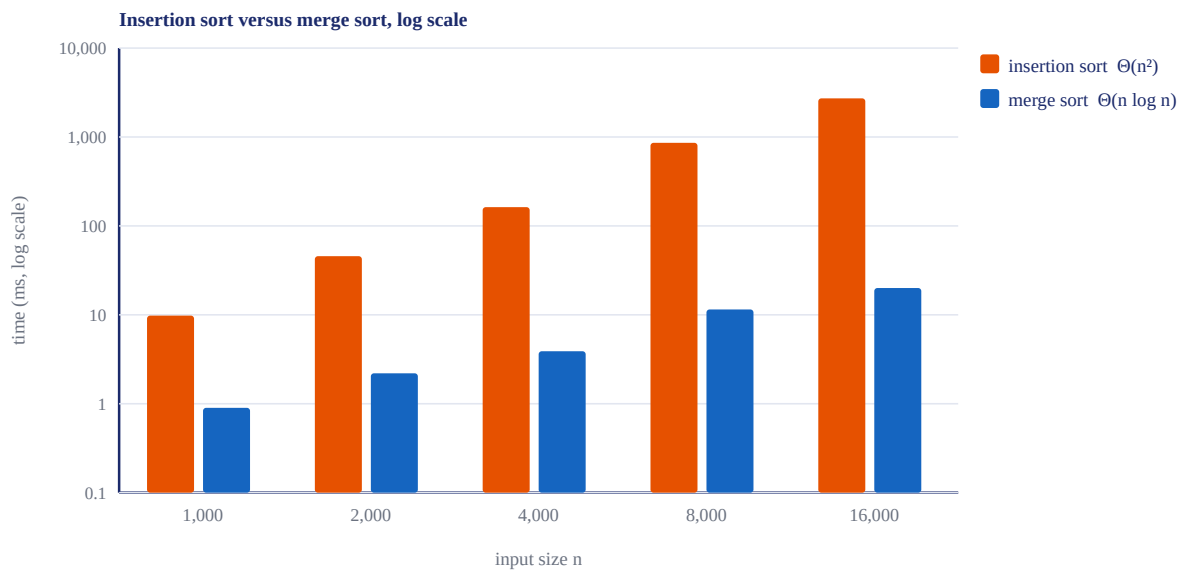
$\Theta(n)$ auxiliary memory. The merged output cannot be written back into the space it came from, so a second array is unavoidable. Insertion sort needs $\Theta(1)$. On a device sorting a million records in limited RAM, that is a real constraint, not a footnote.

4 • Measuring the Claim 12 min

MEASURED ON THE LECTURE MACHINE

SORTING (random input, milliseconds)

n	insertion sort	merge sort	ratio
1,000	9.8	0.9	11x
2,000	45.6	2.2	21x
4,000	162.7	3.9	42x
8,000	860.9	11.5	75x
16,000	2,721.9	20.0	136x



The doubling test on real data. Insertion sort roughly quadruples at each doubling of n ; merge sort roughly doubles. On a log scale a quadratic climbs twice as steeply as a linearithmic one, and the widening gap between the bars is this course's central argument, measured rather than asserted.

Two honest observations. First, the advantage is only $11\times$ at $n = 1000$ — real but modest, because merge sort's constant factor is larger. Second, extrapolate one more row: at $n = 10^6$, insertion sort needs roughly three hours and merge sort about two seconds. That gap is not a tuning problem, and no amount of optimisation rescues an exponent.

5 · What Real Libraries Do 10 min

Python's `sorted()` and `list.sort()` use **Timsort**, which is merge sort with two amendments this lecture has already justified:

Insertion sort at the bottom

Below about 64 elements the recursion stops and insertion sort finishes the block, because §3.1 of Session 9 showed it is fastest there. The complexity class does not change; the constant improves.

Exploit runs that already exist

Real data arrives partly ordered. Timsort *finds* existing sorted runs and merges those, rather than dividing blindly — reaching $\Theta(n)$ on already-sorted input: insertion sort's best case, with merge sort's worst-case guarantee.

The lesson to take, and the one not to

Take: asymptotics choose the algorithm; constants decide how it is tuned afterwards.

Do not take: “the library is fast, so analysis is unnecessary.” Timsort exists precisely because someone analysed both algorithms carefully enough to know exactly where each one wins.

6 · A Free Application: Counting Inversions 13 min

An **inversion** is a pair (i, j) with $i < j$ but $A[i] > A[j]$ — a pair that is out of order. The number of inversions measures how far a list is from sorted, and it is the standard way to measure how far two rankings disagree: have two people rank the same ten films, and the inversion count between their lists is a similarity score.

Brute force examines all pairs: $\Theta(n^2)$. But merge sort already visits the structure we need. During a merge, if `right[j]` is emitted while `len(left) - i` elements remain on the left, then every one of those left elements is greater than it — and each of those is an inversion. All of them can be counted in a single addition.

PY SORT_AND_COUNT

$\Theta(n \log n)$ — same tree, one ex

```
def sort_and_count(A):
    """Return (sorted copy, number of inversions) in  $O(n \log n)$ ."""
    if len(A) <= 1:
        return A[:], 0
    mid = len(A) // 2
    left, a = sort_and_count(A[:mid])
    right, b = sort_and_count(A[mid:])

    out, i, j, crossing = [], 0, 0, 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            out.append(left[i])
            i += 1
        else:
            out.append(right[j])
            j += 1
            crossing += len(left) - i  # every remaining left element beats right
    out.extend(left[i:])
    out.extend(right[j:])
    return out, a + b + crossing
```

PYTHON SESSION

```
>>> sort_and_count([2, 4, 1, 3, 5])
([1, 2, 3, 4, 5], 3)  # the inversions are (2,1), (4,1), (4,3)
```

```
>>> sort_and_count([5, 4, 3, 2, 1])
([1, 2, 3, 4, 5], 10)          # reverse order: all 10 pairs are inverted

>>> sort_and_count(list(range(1000)))
([0, 1, 2, ...], 0)           # already sorted: no inversions
```

THE PATTERN WORTH STEALING

We did not invent a new algorithm. We noticed that merge sort already touches every pair of elements that straddles a split, exactly once, and we charged a counter for it. Whenever a divide-and-conquer algorithm already visits the structure you care about, a second answer is often available for free. Look for this.

Connect it back

The maximum possible number of inversions is $n(n-1)/2$ — the triangle from Session 5, achieved by a reverse-sorted list. And insertion sort's running time is exactly the number of inversions plus n , which explains both its best case and its worst case in a single sentence.

SESSION 11 TAKEAWAY

Divide, conquer, combine. Merge sort pairs a trivial divide with a clever linear combine and pays $T(n) = 2T(n/2) + n = \Theta(n \log n)$ on every input, at a cost of $\Theta(n)$ extra memory. Insertion sort is $\Theta(n^2)$ but wins on small and nearly sorted input — which is why the two live together inside every serious sorting library.

IN-CLASS EXERCISES (15 min — work in pairs)

1. Trace `merge_sort([38, 27, 43, 3, 9, 82, 10])`. Draw the full tree, showing the sublist at every node on the way down and the merged result on the way up.
2. How many comparisons does `merge` make in the best case for two lists of length $n/2$? In the worst case? Give an input achieving each.
3. `merge_sort` uses `A[:mid]` rather than index bounds. What does that slicing cost at each node, and does it change the Θ ? (Careful — think about the total across a whole level.)
4. Modify `merge` to sort descending. Which single character changes? Is the result still stable, and stable with respect to what?
5. A list of n elements has at most how many inversions? Which list achieves it? What does that tell you about insertion sort's worst case?

Homework · due Tuesday 27 October (Workshop II)

1. Implement `merge` and `merge_sort`. Verify against `sorted()` on 1000 random lists, then reproduce the timing table against your insertion sort from Session 9 for $n = 1000 \dots 16000$. Submit code, table and a plot.
2. Re-run both on already-sorted and on reverse-sorted input. Explain why one algorithm's numbers move dramatically and the other's barely move at all.
3. Find the value of n on *your* machine where merge sort overtakes insertion sort. Then build the hybrid — merge sort switching to insertion sort below a threshold k — and tune k . Report your k and the speed-up over plain merge sort.
4. Implement inversion counting both ways. Check they agree on 500 random lists, and time both at $n = 1000, 2000, 4000, 8000$.
5. Demonstrate that changing `<=` to `<` in `merge` breaks stability: construct an input of (name, grade) pairs where the two versions give different output, both correctly sorted by grade.
6. **Bring to Workshop II:** your list of anything from Sessions 8–11 you cannot yet do without help. The workshop is built around those, and the midterm is three days later.

Next session · Tuesday 27 October — Workshop II: No new material. Stations on recursion, recurrences, trees and merge sort, and a full walk-through of what the midterm on Friday 30 October will and will not ask.